

UNIVERSITY OF BRISTOL

Summer 2025 Examination Period

School of Computer Science

1st Year Examination for the Degrees of
Bachelor and Master of Engineering, and Bachelor and Master of
Engineering in Computer Science

COMS10015
Computer Architecture

TIME ALLOWED:
2 Hours

Answers

General Instructions

1. Make sure you carefully read any instructions on the question sheet and answer booklet.
2. You must use the answer booklet to answer all questions.
3. All answers should be short, comprising of at most a few sentences.
4. If the solution requires calculation, you should include all intermediate steps (or “working out”) in your answer.
5. A standard, non-programmable calculator may be used. It must comply with Faculty regulations.
6. This is a *closed book* examination; books, notes, or other resources are forbidden.
7. Any figures or tables referred to can be found towards the end of the question sheet.
8. Any empty pages at the end of the question sheet can be used for rough work, but *only* the answer booklet will be marked.

TURN OVER ONLY WHEN TOLD TO START WRITING

Question 1 (30 marks)

Classify each of the following statements as either **true** or **false**, then explain why using at most a few sentences; omitting either the classification or explanation will result in zero marks.

- (a) A Boolean function $f : \{0, 1\}^2 \rightarrow \{0, 1\}$ will require at least 1 Boolean operator to implement it. (3 marks)

Solution: false. Notice that f is a 2-input, 1-output Boolean function. Since it has $n = 2$ inputs, there are $2^{2^n} = 2^{2^2} = 16$ possible functions, some of which are as follows:

x	y	f_0	f_{10}	f_{12}	f_{15}
0	0	0	0	0	1
0	1	0	1	0	1
1	0	0	0	1	1
1	1	0	1	1	1

These could be described as a “constant 0” function $f_0(x, y) = 0$, an “identity” function $f_{10}(x, y) = y$, an “identity” function $f_{12}(x, y) = x$, and a “constant 1” function $f_{15}(x, y) = 1$ respectively; they can all be implemented using 0 Boolean operators.

- (b) The organisation of N- and P-type MOSFET transistors shown in Figure 1 (on Page 9) implements the Boolean expression $r = (x \wedge \neg c) \vee (y \wedge c)$, i.e., that reflecting a 2-input, 1-bit multiplexer. (3 marks)

Solution: false. Given that c , x , and y are all connected to transistor gate terminals, they represent inputs; r is not, so is therefore an output. So, as a first step or a sanity check, this does indeed match the expression. Therefore, using $V_{dd} \equiv 1$ and $V_{ss} \equiv 0$, we can describe the behaviour exhibited by considering two cases:

1. If $c = 0$ then either the top-left path from V_{dd} or bottom-left path from V_{ss} is possible, depending on x : so if $c = 0$ and $x = 0$ then $r = 1$ (via the top-left path, through bottom P-MOSFET because $c = 0$ and top P-MOSFET because $x = 0$) whereas if $c = 0$ and $x = 1$ then $r = 0$ (via the bottom-left path, through top N-MOSFET because $\neg c = 1$ and bottom N-MOSFET because $x = 1$).
2. If $c = 1$ then either the top-right path from V_{dd} or bottom-right path from V_{ss} is possible, depending on y : so if $c = 1$ and $y = 0$ then $r = 1$ (via the top-right path, through bottom P-MOSFET because $\neg c = 0$ and top P-MOSFET because $y = 0$) whereas if $c = 1$ and $y = 1$ then $r = 0$ (via the bottom-right path, through top N-MOSFET because $c = 1$ and bottom N-MOSFET because $y = 1$).

In summary, this means

c	x	y	r
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

and therefore $r = \neg((x \wedge \neg c) \vee (y \wedge c))$ meaning it implements a “multiplexer with inverted output”.

- (c) A full-adder is functionally complete (or universal), in the sense that one can realise *all* standard Boolean operators (i.e., NOT, AND, and OR) and therefore *any* Boolean expression using instances of it alone. (3 marks)

Solution: true. Recall that the truth table for a full-adder (i.e., 1-bit adder) reads as follows

ci	x	y	co	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

This captures the fact that it computes two 1-bit outputs (i.e., the carry-out co and sum s) from three 1-bit inputs (i.e., the carry-in ci and addends x and y). Imagine that $+(ci, x, y)_{co}$ and $+(ci, x, y)_s$ respectfully denote the co and s outputs of a full-adder, as applied to the inputs ci , x and y : one can see that

$$\begin{aligned} x \wedge y &\equiv +(0, x, y)_{co} \\ x \vee y &\equiv +(1, x, y)_{co} \\ \neg y &\equiv +(0, 1, y)_s \end{aligned}$$

which implies said the full-adder is functionally complete in the sense required.

- (d) SRAM and DRAM memories are both classed as volatile. However, a DRAM cell will retain the value stored for a short period after being powered-off, whereas an SRAM cell will not. (3 marks)

(cont.)

Solution: true. In essence, the reason for this is that the value stored by a DRAM is maintained in a capacitor: if the capacitor is charged the value is 1, whereas if the capacitor is dis-charged the value is 0. Even when powered-on that charge has to be periodically refreshed because it will degrade (or decay); when powered-off the same sort of degradation occurs, likely at a faster rate, until the charge and value is lost. But in neither case is the degradation instantaneous. So, for a short period after a DRAM cell has been powered-off, it will retain some decaying amount of charge and so the value stored.

- (e) Although Moore- and Mealy-style Finite State Machines (FSMs) have different structures, every instance of the former can be converted into (or restructured to form) an instance of the latter. (3 marks)

Solution: true. Let $\delta_M(\cdot)$ and $\omega_M(\cdot)$ respectively denote the transition and output functions for some FSM labelled M . Every Moore-style FSM X can be converted into a Mealy-style FSM Y , by defining $\delta_Y(Q, x) = \delta_X(Q, x)$ and $\omega_Y(Q, x) = \omega_X(\delta_X(Q, x))$. That is, the input x is accommodated in ω_Y by 1) applying δ_X to “look ahead” at the next state, then 2) applying ω_X to produce the output for said state. However, not every Mealy-style FSM can be converted into a Moore-style FSM: some can, but even then with a “shift” in when the output is produced.

- (f) A Translation Lookaside Buffer improves the performance of memory access by storing recently used virtual to physical address mappings. (3 marks)

Solution: true. The mapping stored in the TLB caches mappings from the Page Table. The Page Table is stored in memory and so without the TLB, two accesses to memory are required (1 mark). Access to the Page Table is quicker than accessing main memory, so even though the processor still has to do the translation, the overall latency is lower when there is a TLB (1 mark).

- (g) A load instruction accessed element i of an array A (i.e., $A[i]$), which did not appear in L1 cache. A small number of instructions later, a subsequent load instruction accesses element $i + 1$ of the array A (i.e., $A[i + 1]$), which does appear in L1 cache due to the property of temporal locality. (3 marks)

Solution: false. Assuming that elements i and $i + 1$ of the array A are to be found on a cache line (which is likely given the question states that the access $A[i + 1]$ is a cache hit), then it is *spatial* locality, and not *temporal* locality, that meant the access was a cache hit.

- (h) Consider a processor with two levels of data cache, L1 and L2, where L2 is inclusive of L1. When a cache line is evicted from the L1 cache, a copy of that cache line may still be found in the L2 cache, providing a performance benefit for further L1 cache misses for that cache line. (3 marks)

Solution: true. An inclusive cache policy means that when a load misses L1 cache, the data is placed in both levels of cache such that L2 contains a copy of the data in L1, plus additional data from previous evictions (and perhaps additional data in the presence of L2 prefetching). As such, when data leaves the L1 cache, it may still remain in L2, which would yield a performance benefit for a subsequent L1 miss as it can be fulfilled from L2 instead of more distant memory.

Award one mark for correct explanation of an inclusive cache policy, and award one mark for explaining how the performance is improved for subsequent L1 misses in this regime.

- (i) Consider an instruction set architecture (ISA) that uses variable length encoding. The program counter register must be incremented by one during the execution cycle so that it contains the address of the next instruction in memory to subsequently fetch. (3 marks)

Solution: false. Instructions may be of differing lengths due to variable length encoding, and so the program counter needs to be incremented by a variable amount each time, not by a fixed offset.

- (j) Caches are only used in a processor as a means of reducing the latency of load and store instructions (which access data in memory). (3 marks)

Solution: false. There are typically also caches for instructions and the page table (i.e., the TLB).

Question 2 (3 marks)

Explain the difference between an architectural register and a micro-architectural register. Provide one example of each type of register.

Solution: An architectural register is one defined in the Instruction Set Architecture, and forms part of the processor state. For example, these registers can be a destination/source for instructions.

A micro-architectural register does not appear in the ISA, and is an implementation detail, and as such cannot be used in any instructions. For example, micro-architectural registers may be used to store intermediate results between pipeline stages.

One mark is available for a correct definition of both architectural and micro-architectural registers. One mark is available for a correct example of each.

Question 3 (3 marks)

Give an example of a source of a hardware interrupt. Explain how, and when, the processor handles the interrupt.

Solution: Hardware interrupts occur at anytime in the execution cycle and must be handled as soon as possible. The micro-architecture will often check during the Fetch stage of the execution cycle if an exception/interrupt has occurred. When an exception is detected, the processor then jumps according to the interrupt vector to the interrupt handler.

Award one mark for “when” (during Fetch), one mark for “how” (interrupt vector/handler), and one mark for any correct example of an external hardware interrupt (e.g., keyboard press), or an internal hardware interrupt (e.g., timing interrupt).

Question 4 (3 marks)

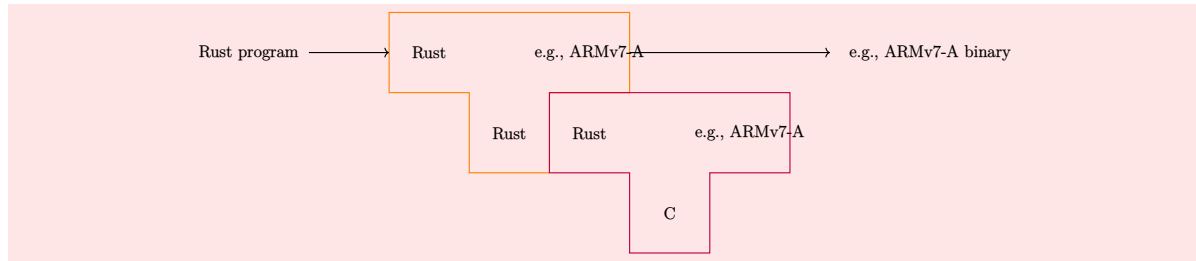
Draw a Tombstone diagram (T-diagram) to show how a new Rust compiler (written in Rust) can be bootstrapped using the existing GNU GCC compiler for C. You can target any Instruction Set Architecture you like, such as IA-32, ARMv7-A, etc.

Solution:

Award one mark for correctly showing that the first compiler translates Rust using Rust.

Award one mark for correctly showing that the first compiler outputs a binary.

Award one mark for correctly showing the structure of the second compiler.



Question 5 (6 marks)

- (a) Calculate the runtime of the program described by Table 1 (on Page 10) according to the Iron Law of Processor Performance when executed on a 2 GHz (2×10^9 Hz) processor. Note that the Iron Law is shown in Figure 2 (on Page 10). (3 marks)

Solution: There are $(20 + 10 + 20 + 10 + 10) \times 10^6 = 70 \times 10^6$ instructions in the program — this corresponds to the first term of Iron Law, and the denominator in the cycles-per-instruction (CPI) calculation. Award one mark for correct calculation.

The CPI is found by multiplying the table rows, summing the products, and dividing by the total number of instructions:

$$\frac{((4 \times 20) + (4 \times 10) + (1 \times 20) + (2 \times 10) + (1 \times 10)) \times 10^6}{70 \times 10^6} = \frac{170 \times 10^6}{70 \times 10^6} (\approx 2.429)$$

Note that it is better to leave it in rational form. Award one mark for correct calculation of CPI.

The runtime of the program is then calculated under the Iron Law as:

$$\begin{aligned} \text{runtime} &= (70 \times 10^6) \times \frac{170 \times 10^6}{70 \times 10^6} \times \frac{1}{2 \times 10^9} \\ &= \frac{170 \times 10^6}{2 \times 10^9} \\ &= 85 \times 10^{-3} \\ &= 0.085 \text{ seconds} \end{aligned}$$

Award one mark for correct application of Iron Law with the previously calculated terms.

- (b) Describe one way that the performance of the program might be improved, along with any limitations of that approach. You should state which term in the Iron Law the improvement would change. (3 marks)

(cont.)

Solution: Award 1 mark for a correct performance improvement, 1 mark for the limitations, and 1 mark for identifying the term in the Iron Law.

The clock speed (frequency) of the processor could be increased, allowing each instruction to take less time to execute. This is limited by the power and cooling of the processor which is hard due to the limitations imposed by the Power Wall. This affects (reduces) the final term in the Iron Law.

The middle term, CPI, can be improved by pipelining a scalar processor. This would improve this term such that in the best case it was equal to 1 cycle per instruction where there were no bubbles in the pipeline, however this would be unlikely in practice.

The first term, instructions per program, might be improved by using a CISC Instruction Set, where there are fewer instructions necessary to implement the program. However, this will also likely impact on the second term (CPI), increasing the number of cycles needed for each instruction, and so there would need to be a careful balance of the terms to yield a performance improvement in practice.

Note: It would be unrealistic to improve the cycle counts of the instructions themselves given the knowledge of the memory hierarchy (loads of only a few cycles likely satisfied from cache). Likewise, any argument around using a RISC ISA along the lines of the inverse of the CISC argument above is not sufficiently compelling to award marks.

Additional figures and tables

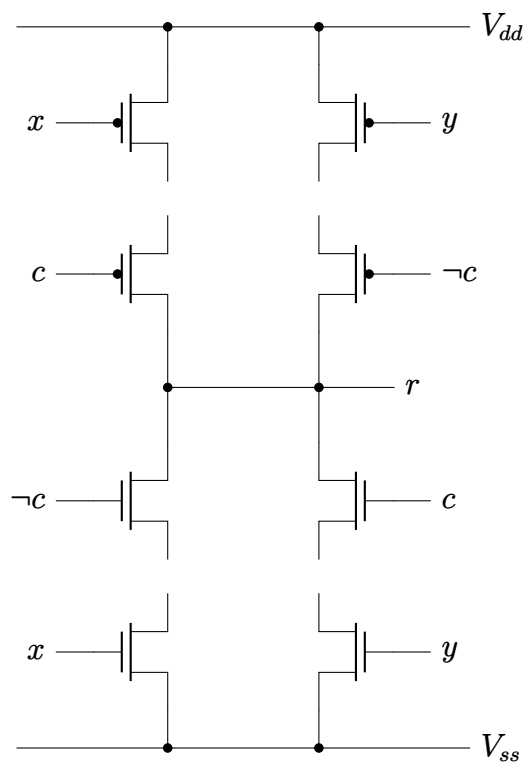


Figure 1: A design that uses MOSFET-based transistors, with three inputs (labelled c , x , and y) and one output (labelled r).

(cont.)

Instruction	Cycles per instruction	Occurrences
Loads	4 cycles	20×10^6
Stores	4 cycles	10×10^6
Add	1 cycle	20×10^6
Branch	2 cycles	10×10^6
Logical comparison	1 cycle	10×10^6

Table 1: The instruction mix of a program.

$$\text{Execution Time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

Figure 2: Iron Law of Processor Performance.